

Herencia, Atributos, Métodos y Polimorfismo (POO)

1. ¿Qué es la Herencia?

La herencia permite que una clase hija herede los atributos y métodos de una clase padre.

Ventajas:

- Reutiliza código.
- Evita repetir información.
- Facilita el mantenimiento del programa.

2. Atributos

Son las características o datos de un objeto.

Ejemplo:

```
class Animal {  
    String nombre;
```

Java

Aquí:

- nombre y edad son atributos.

3. Métodos

Son las acciones que puede realizar un objeto.

Ejemplo:

```
class Animal {  
    void hacerSonido() {  
        System.out.println("El animal hace un sonido");  
    }  
}
```

Java

Aquí:

- `hacerSonido()` es un método.

4. Herencia en Java

```
class Animal {  
    String nombre;  
  
    void comer() {  
        System.out.println("Está comiendo");  
    }  
}
```

```
class Perro extends Animal {  
    void ladrar() {  
        System.out.println("Guau Guau");  
    }  
}
```

Java

Uso:

```
Perro p = new Perro();  
p.nombre = "Max";
```

Java

5. Polimorfismo

El **polimorfismo** permite que un mismo método tenga diferentes comportamientos según el objeto que lo utilice.

Ejemplo:

```
class Animal {  
    void hacerSonido() {  
        System.out.println("Sonido genérico");  
    }  
}  
  
class Perro extends Animal {  
    @Override  
    void hacerSonido() {  
        System.out.println("Guau Guau");  
    }  
}  
  
class Gato extends Animal {  
    @Override  
    void hacerSonido() {  
        System.out.println("Miau");  
    }  
}
```

Java

Programa principal:

```
public class Main {  
    public static void main(String[] args) {  
  
        Animal a1 = new Perro();  
        Animal a2 = new Gato();  
  
        a1.hacerSonido();  
        a2.hacerSonido();  
    }  
}
```

Java

Salida:

```
Guau Guau  
Miau
```

Resumen

- **Atributos:** características de un objeto (nombre , edad).
- **Métodos:** acciones que realiza (comer() , ladrar()).
- **Herencia:** una clase hija recibe atributos y métodos de una clase padre.
- **Polimorfismo:** un mismo método puede comportarse de distintas maneras según el objeto que lo ejecute.



Please Aldo the constructor

Constructor en Herencia

Un **constructor** es un método especial que se ejecuta cuando se crea un objeto. Tiene el mismo nombre de la clase.

Ejemplo completo

```
class Animal {
    String nombre;
    int edad;

    // Constructor
    Animal(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    void hacerSonido() {
        System.out.println("Sonido genérico");
    }
}

class Perro extends Animal {

    // Constructor de la clase hija
    Perro(String nombre, int edad) {
        super(nombre, edad); // llama al constructor de Animal
    }

    @Override
    void hacerSonido() {
        System.out.println("Guau Guau");
    }
}
```

```
}

public class Main {
    public static void main(String[] args) {

        Perro p = new Perro("Max", 3);

        System.out.println("Nombre: " + p.nombre);
        System.out.println("Edad: " + p.edad);

        // herencia
    }
}
```

Java

Salida

```
Nombre: Max
Edad: 3
Guau Guau
```

Explicación

- `Animal(String nombre, int edad)` es el constructor de la clase padre.
- `Perro(String nombre, int edad)` es el constructor de la clase hija.
- `super(nombre, edad)` llama al constructor de la clase padre para inicializar los atributos heredados.
- `@Override` indica que el método de la clase hija reemplaza al de la clase padre (polimorfismo).



🔒 Los mensajes a partir de aquí solo son visibles para ti

Please, let this include abstraction, aggregation, polymorphism, method, attribute, dependency, and encapsulation.

Object-Oriented Programming (OOP)

1. Abstraction

Abstraction means showing only the essential information and hiding unnecessary details.

```
abstract class Animal {
```

Java

`Animal` is an abstract class because it represents a general concept of an animal.

2. Encapsulation

Encapsulation protects data by making attributes private and accessing them through methods.

```
class Animal {  
    private String name;  
  
    public Animal(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Java

- **Attribute:** `name`
- **Methods:** `getName()` and `setName()`

3. Inheritance

Inheritance allows a class to inherit attributes and methods from another class.

```
class Dog extends Animal {  
  
    public Dog(String name) {  
        super(name);  
    }  
}
```

Java

```
}  
}
```

Dog inherits the attribute `name` and the methods from `Animal`.

4. Polymorphism

Polymorphism allows the same method to behave differently in different classes.

```
abstract class Animal {  
    public abstract void makeSound();  
}  
  
class Dog extends Animal {  
  
    public Dog(String name) {  
        super(name);  
    }  
  
    @Override  
    public void makeSound() {  
        System.out.println("Woof Woof");  
    }  
}  
  
class Cat extends Animal {  
  
    public Cat(String name) {  
        super(name);  
    }  
  
    @Override  
    public void makeSound() {  
        System.out.println("Meow");  
    }  
}
```

Java

Example:

```
Animal animal1 = new Dog("Max");  
Animal animal2 = new Cat("Luna");
```

Java

Output:

```
Woof Woof  
Meow
```

5. Aggregation

Aggregation is a "has-a" relationship where one object contains another object, but both can exist independently.

```
class Owner {  
    private String ownerName;  
  
    public Owner(String ownerName) {  
        this.ownerName = ownerName;  
    }  
}  
  
class Dog extends Animal {  
    private Owner owner;  
  
    public Dog(String name, Owner owner) {  
        super(name);  
        this.owner = owner;  
    }  
}
```

Java

A Dog has an Owner, but the owner can exist without the dog.

6. Dependency

Dependency occurs when one class uses another class temporarily.

```
class Veterinarian {  
  
    public void examine(Animal animal) {  
        System.out.println("Examining " + animal.getName());  
    }  
}
```

Java

Veterinarian depends on Animal because it uses it as a parameter.

Complete Example

```
public class Main {  
    public static void main(String[] args) {  
  
        Owner owner = new Owner("John");  
  
        Animal dog = new Dog("Max", owner);  
        Animal cat = new Cat("Luna");  
  
        dog.makeSound();  
        cat.makeSound();  
  
        Veterinarian vet = new Veterinarian();  
        vet.examine(dog);  
    }  
}
```

Java

Summary Table

Concept	Example
Attribute	name , owner
Method	makeSound() , getName()
Constructor	Animal(String name)
Encapsulation	Private attributes with getters/setters
Abstraction	abstract class Animal
Inheritance	Dog extends Animal
Polymorphism	makeSound() behaves differently in Dog and Cat
Aggregation	Dog has an Owner
Dependency	Veterinarian uses Animal

This example includes **attributes**, **methods**, **constructors**, **encapsulation**, **abstraction**, **inheritance**, **polymorphism**, **aggregation**, and **dependency** in one Java program.



give me in a code

Complete Java Code with Abstraction, Encapsulation, Inheritance, Polymorphism, Aggregation, Dependency, Attributes, Methods, and Constructors

```
public class Main {  
    public static void main(String[] args) {  
  
        Owner owner = new Owner("John");  
  
        Animal dog = new Dog("Max", 3, owner);  
        Animal cat = new Cat("Luna", 2);  
  
        dog.makeSound();  
        cat.makeSound();  
  
        Veterinarian vet = new Veterinarian();  
        vet.examine(dog);  
        vet.examine(cat);  
    }  
}  
  
// Abstraction  
abstract class Animal {  
  
    // Attributes  
    private String name;  
    private int age;  
  
    // Constructor  
    public Animal(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // Encapsulation (Getters and Setters)  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

```
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    // Abstract Method
    public abstract void makeSound();
}

// Inheritance + Aggregation
class Dog extends Animal {

    private Owner owner; // Aggregation

    public Dog(String name, int age, Owner owner) {
        super(name, age);
        this.owner = owner;
    }

    public Owner getOwner() {
        return owner;
    }

    // Polymorphism
    @Override
    public void makeSound() {
        System.out.println(getName() + " says: Woof Woof!");
    }
}

// Inheritance
class Cat extends Animal {

    public Cat(String name, int age) {
        super(name, age);
    }

    // Polymorphism
    @Override
    public void makeSound() {
        System.out.println(getName() + " says: Meow!");
    }
}

// Aggregation Class
class Owner {

    private String ownerName;
```

```

    public Owner(String ownerName) {
        this.ownerName = ownerName;
    }

    public String getOwnerName() {
        return ownerName;
    }
}

// Dependency
class Veterinarian {

    public void examine(Animal animal) {
        System.out.println(
            "Examining " + animal.getName() +
            ", Age: " + animal.getAge()
        );
    }
}

```

Java

Where each concept appears

Concept	Code Example
Attribute	name , age , owner , ownerName
Method	makeSound() , getName() , examine()
Constructor	Animal() , Dog() , Cat() , Owner()
Encapsulation	Private attributes + getters/setters
Abstraction	abstract class Animal
Inheritance	Dog extends Animal , Cat extends Animal
Polymorphism	makeSound() overridden in Dog and Cat
Aggregation	Dog has an Owner
Dependency	Veterinarian uses Animal as a parameter

Output

```

Max says: Woof Woof!
Luna says: Meow!

```

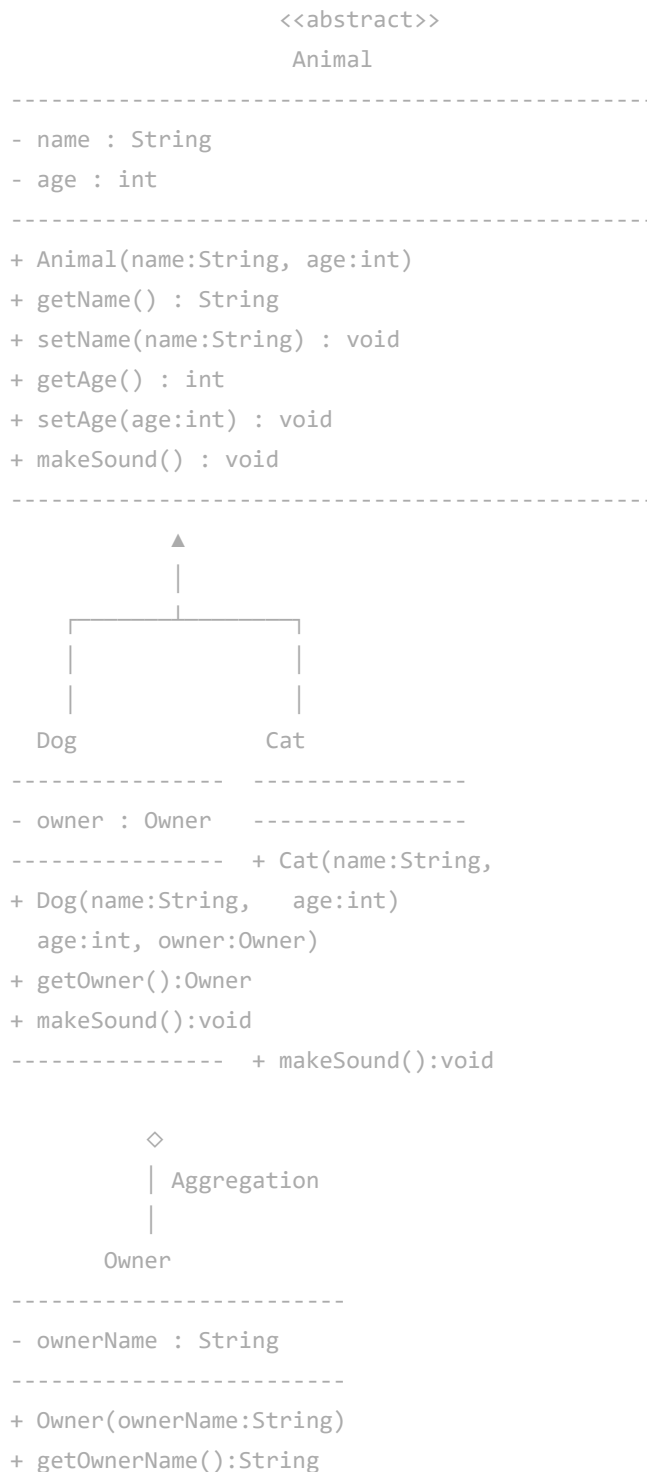
Examining Max, Age: 3

Examining Luna, Age: 2



uml

Aquí tienes el **UML Class Diagram** para el código anterior:



```
-----
Veterinarian
-----
```

```
+ examine(animal:Animal)
  : void
-----
```

```
Veterinarian -----> Animal
      Dependency
```

Relationships

- **Inheritance (Generalization)**
 - Dog → Animal
 - Cat → Animal
- **Aggregation**
 - Dog ◇— Owner
 - A dog has an owner, but the owner can exist without the dog.
- **Dependency**
 - Veterinarian -----> Animal
 - The veterinarian uses an animal as a parameter in `examine()` .

PlantUML Version

```
@startuml
```

```
abstract class Animal {
    - name : String
    - age : int
    + Animal(name:String, age:int)
    + getName() : String
    + setName(name:String) : void
    + getAge() : int
    + setAge(age:int) : void
    + makeSound() : void
}
```

```
class Dog {
    - owner : Owner
    + Dog(name:String, age:int, owner:Owner)
    + getOwner() : Owner
    + makeSound() : void
}
```

```
class Cat {
```

```

+ Cat(name:String, age:int)
+ makeSound() : void
}

class Owner {
- ownerName : String
+ Owner(ownerName:String)
+ getOwnerName() : String
}

class Veterinarian {
+ examine(animal:Animal) : void
}

Animal <|-- Dog
Animal <|-- Cat
Dog o-- Owner
Veterinarian --> Animal

plantuml

```

Este UML muestra **abstraction**, **encapsulation** (atributos privados), **inheritance**, **polymorphism**, **aggregation**, **dependency**, **methods**, **attributes** y **constructors**.



please allow what the interface this el abstraction yes is so
give me as four example

Yes. In Java, **abstraction** can be implemented in **two ways**:

1. **Abstract Class**
2. **Interface**

If your teacher asks for **4 examples of abstraction**, you can use these:

1. Abstraction using an Abstract Class

```

abstract class Animal {

    public abstract void makeSound();
}

```

```
}
```

```
class Dog extends Animal {  
  
    @Override  
    public void makeSound() {  
  
        Java  
    }  
}
```

Abstraction: Animal

Implementation: Dog

2. Abstraction using an Interface

```
interface Flyable {  
  
    void fly();  
  
}  
  
class Bird implements Flyable {  
  
    @Override  
    public void fly() {  
        System.out.println("Bird is flying");  
    }  
  
    Java  
}
```

Abstraction: Flyable

Implementation: Bird

3. Abstraction using an Abstract Class

```
abstract class Vehicle {  
  
    public abstract void move();  
  
}  
  
class Car extends Vehicle {
```



```
@Override
public void move() {
    System.out.println("Car is moving");
}
```

Java

Abstraction: Vehicle

Implementation: Car

4. Abstraction using an Interface

```
interface Payment {

    void pay(double amount);

}

class CreditCard implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount);
    }
}
```

Java

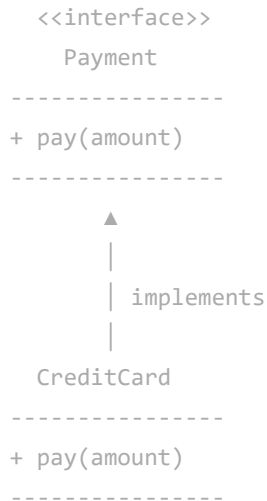
Abstraction: Payment

Implementation: CreditCard

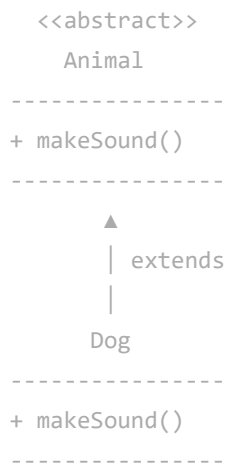
Summary

Example	Type of Abstraction	Implementation
Animal	Abstract Class	Dog
Flyable	Interface	Bird
Vehicle	Abstract Class	Car
Payment	Interface	CreditCard

UML Example (Interface)



UML Example (Abstract Class)



These four examples clearly demonstrate **abstraction** using both **abstract classes** and **interfaces** in Java.



in a code and explain

Abstraction in Java (Using an Interface)

Code Example

```
// Abstraction with Interface
interface Payment {
```

```

    void pay(double amount);
}

// Implementation 1
class CreditCard implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using Credit Card");
    }
}

// Implementation 2
class PayPal implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using PayPal");
    }
}

public class Main {
    public static void main(String[] args) {

        Payment payment1 = new CreditCard();
        Payment payment2 = new PayPal();

        payment1.pay(100);
        payment2.pay(50);
    }
}

```

Java

Explanation

1. Interface (Abstraction)

```
interface Payment {
```

Java

- Payment is an **interface**.
- It defines **what** a payment must do (pay()).
- It does not specify **how** the payment is made.
- This is **abstraction** because it hides the implementation details.

2. Implementation Classes

Java

Java

- Both classes implement the `Payment` interface.
- Each class provides its own version of the `pay()` method.

3. Polymorphism

Java

- The reference type is `Payment`.
- The actual objects are `CreditCard` and `PayPal`.
- The same method `pay()` behaves differently depending on the object.

4. Method

Java

- `pay()` is a method.
- It performs the payment action.

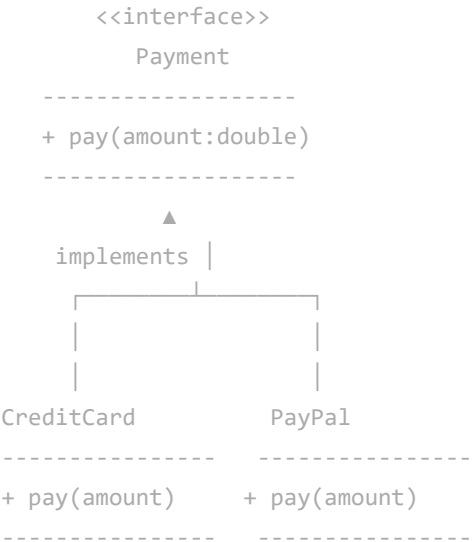
5. Output

```
Paid $100.0 using Credit Card  
Paid $50.0 using PayPal
```

Concepts in this Example

Concept	Example
Abstraction	<code>interface Payment</code>
Method	<code>pay(double amount)</code>
Polymorphism	<code>Payment payment1 = new CreditCard();</code>
Interface	<code>Payment</code>
Implementation	<code>CreditCard , PayPal</code>

UML Diagram



UML Explanation

- `Payment` is an interface (**abstraction**).
- `CreditCard` and `PayPal` implement the interface.
- Both classes provide their own implementation of the `pay()` method.
- This demonstrates **abstraction** and **polymorphism** in Java.



this is all

No. The previous code mainly demonstrates **Abstraction, Interface, Method, and Polymorphism**.

If your teacher asks to include **Abstraction, Encapsulation, Inheritance, Polymorphism, Attributes, Methods, Constructors, Aggregation, and Dependency**, you need a more complete example like this:

```
// Abstraction
interface Payment {
    void pay(double amount);
}

// Encapsulation
class Customer {
    private String name; // Attribute

    // Constructor
    public Customer(String name) {
        this.name = name;
    }

    // Getter
    public String getName() {
        return name;
    }
}

// Inheritance
class OnlinePayment {
    protected String transactionId;

    public OnlinePayment(String transactionId) {
        this.transactionId = transactionId;
    }
}

// Inheritance + Interface Implementation + Aggregation
class CreditCard extends OnlinePayment implements Payment {

    private Customer customer; // Aggregation

    public CreditCard(String transactionId, Customer customer) {
        super(transactionId);
        this.customer = customer;
    }
}
```

```
@Override
public void pay(double amount) {
    System.out.println(customer.getName()
        + " paid $" + amount
        + " with Credit Card.");
}

}

// Dependency
class PaymentProcessor {

    public void processPayment(Payment payment, double amount) {
        payment.pay(amount);
    }
}

public class Main {
    public static void main(String[] args) {

        Customer customer = new Customer("Odalys");

        Payment payment = new CreditCard("TX001", customer);

        PaymentProcessor processor = new PaymentProcessor();

        processor.processPayment(payment, 100);
    }
}
```

Java

Explanation of the Concepts

Concept	Example in the Code
Attribute	name , transactionId , customer
Method	pay() , getName() , processPayment()
Constructor	Customer() , OnlinePayment() , CreditCard()
Abstraction	interface Payment
Encapsulation	private String name with getName()
Inheritance	CreditCard extends OnlinePayment
Polymorphism	Payment payment = new CreditCard(...)
Aggregation	CreditCard has a Customer

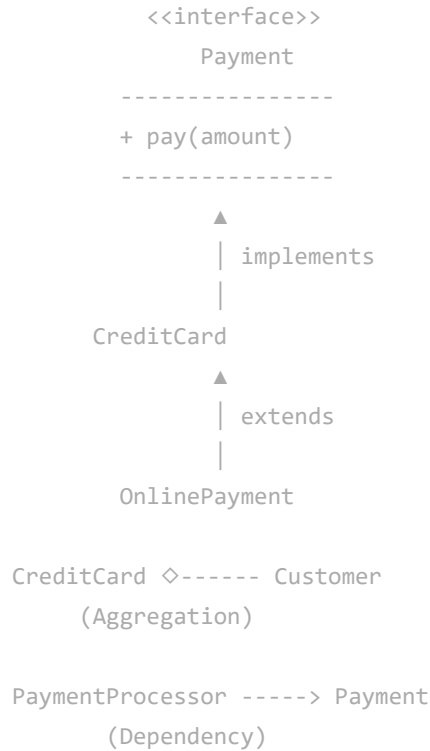
Concept

Example in the Code

Dependency

PaymentProcessor uses Payment

UML



This is a complete OOP example that includes all the concepts usually required in Unit 2: **Abstraction, Encapsulation, Inheritance, Polymorphism, Attributes, Methods, Constructors, Aggregation, and Dependency.**



ahora en netbeans

Si lo vas a hacer en **NetBeans**, lo recomendable es crear una clase por archivo dentro del paquete `model` y una clase `Main` para ejecutar el programa.

Estructura del proyecto

```

src
├── ec.edu.espe.paymentsystem
│   ├── Payment.java
│   └── Customer.java
  
```



```
|— OnlinePayment.java  
|— CreditCard.java  
|— PaymentProcessor.java  
|— Main.java
```

1. Payment.java (Interface - Abstraction)

```
package ec.edu.espe.paymentsystem;  
  
public interface Payment {  
  
    Java
```

2. Customer.java (Encapsulation)

```
package ec.edu.espe.paymentsystem;  
  
public class Customer {  
    private String name;  
  
    public Customer(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    Java
```

3. OnlinePayment.java (Parent Class)

```
package ec.edu.espe.paymentsystem;

public class OnlinePayment {
    protected String transactionId;

    public OnlinePayment(String transactionId) {
        this.transactionId = transactionId;
    }

    Java
    ,
}
```

4. CreditCard.java (Inheritance + Polymorphism + Aggregation)

```
package ec.edu.espe.paymentsystem;

public class CreditCard extends OnlinePayment implements Payment {
```

 **ChatGPT** ▾  **Mejorar el plan**

 ...

```
    public CreditCard(String transactionId, Customer customer) {
        super(transactionId);
        this.customer = customer;
    }
```

```
@Override
public void pay(double amount) {
    System.out.println(customer.getName()
        + " paid $" + amount
        + " using Credit Card.");
}
```

Java

5. PaymentProcessor.java (Dependency)

```
package ec.edu.espe.paymentsystem;

public class PaymentProcessor {

    public void processPayment(Payment payment, double amount) {
```

```
payment.pay(amount);
```

Java

6. Main.java

Java

```
package ec.edu.espe.paymentsystem;

public class Main {

    public static void main(String[] args) {

        Customer customer = new Customer("Odalys");

        Payment payment = new CreditCard("TX001", customer);

        PaymentProcessor processor = new PaymentProcessor();

        processor.processPayment(payment, 150.00);

    }

}
```

Salida

```
Odalys paid $150.0 using Credit Card.
```

Dónde está cada concepto

Concepto	Clase
Abstraction	Payment (interface)
Encapsulation	Customer
Inheritance	CreditCard extends OnlinePayment
Polymorphism	Payment payment = new CreditCard(...)
Attribute	name , transactionId , customer
Method	pay() , getName() , processPayment()

Concepto

Clase

Constructor

Todas las clases tienen constructor

Aggregation

CreditCard tiene un Customer

Dependency

PaymentProcessor usa Payment

Este formato es el que normalmente se entrega en **NetBeans** junto con el **diagrama UML**.

